Reactor: Write Up

Plataforma: HackTheBox

Máquina / Desafio:	Reactor
Dificuldade:	Easy
Data de Resolução:	30 de maio de 2026
Autor:	Alan Dias (alankk)

31 de maio de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**
2.1 Varredura de Portas – Nmap 2
2.2 Enumeração de Serviços – Porta 3000 3

3 Exploração **4**
3.1 Identificação da Versão do Next.js 4
3.2 A Vulnerabilidade – CVE-2025-66478 (React2Shell) 4
3.3 Exploração – react2shell-ultimate 4

4 Pós-Exploração – Enumeração Local **5**
4.1 Descoberta do Banco de Dados 5
4.2 Identificação do Tipo de Hash 6
4.3 Quebra de Hash – John the Ripper 6

5 Movimentação Lateral – Acesso SSH **8**

6 Escalada de Privilégios **8**
6.1 Enumeração Local – Serviços Internos 8
6.2 O Vetor – Node.js Inspector Abuse 9
6.3 Exploração – RCE via Node Inspector 9

7 Conclusão **10**

8 Referências **10**

1 Visão Geral

Reactor é uma máquina Linux cuja superfície de ataque inicial é uma aplicação web **Next.js 15.0.3** exposta na porta 3000. A versão vulnerável do *framework* permitiu explorar a **CVE-2025-66478**, uma falha crítica de execução remota de código (RCE) no protocolo *React Server Components* (RSC), obtendo assim uma *shell* inicial no servidor sob o contexto do usuário comum `node`.
A partir do acesso inicial, a enumeração local revelou um banco de dados SQLite contendo *hashes* de credenciais. A quebra do *hash* correspondente ao operador via *John the Ripper* forneceu credenciais válidas que permitiram a movimentação lateral e o estabelecimento de uma sessão persistente via protocolo SSH. Internamente, identificou-se um processo Node.js executando com privilégios de superusuário (`root`) que expunha a flag de depuração `--inspect`. Essa configuração inadequada permitiu o abuso da interface de *debugging* via *Chrome DevTools Protocol* (CDP), culminando na execução de comandos arbitrários e no comprometimento total do ativo.

2 Reconhecimento

2.1 Varredura de Portas – Nmap

O primeiro passo consistiu em realizar uma varredura completa de portas na máquina alvo, utilizando um envio acelerado de pacotes para otimizar o tempo de enumeração dos serviços ativos. O comando executado no utilitário `nmap` foi:

```
nmap -sS -p- --min-rate 5000 -Pn -oN portasReactor.txt <TARGET_IP>
```

As diretivas (*flags*) aplicadas no comando são detalhadas a seguir:

- **Flag `-Pn`:** Desabilita o mecanismo de descoberta de hosts (*host discovery* via ping), garantindo a execução do escaneamento mesmo caso o alvo bloqueie requisições ICMP.
- **Flag `--min-rate 5000`:** Estipula um envio mínimo de 5000 pacotes por segundo, acelerando significativamente o processo de varredura.
- **Flag `-oN`:** Salva o resultado obtido em um arquivo de texto com formato de saída padrão (*normal output*) para referências posteriores.

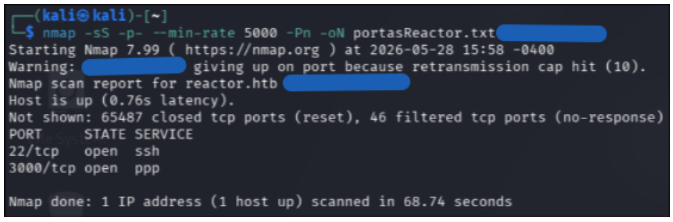


Figura 1: Resultado da varredura inicial: duas portas abertas identificadas.

A varredura inicial revelou duas portas TCP em estado ativo no servidor, conforme sintetizado na Tabela 1:

A porta 22 executa o serviço SSH, consolidando-se como um vetor típico de acesso remoto e persistência após a obtenção de credenciais legítimas. A porta 3000 foi rotulada preliminarmente com o serviço genérico `ppp`, indicando que o `nmap` não reconheceu o protocolo da camada de aplicação através de suas sondagens básicas, o que justifica e demanda uma enumeração mais direcionada.

Tabela 1: Portas e serviços mapeados na varredura inicial.

Porta	Estado	Serviço
22/tcp	open	ssh
3000/tcp	open	ppp

2.2 Enumeração de Serviços – Porta 3000

Para decodificar a tecnologia real em execução por trás da porta 3000, realizou-se uma segunda varredura focada na coleta detalhada de banners e identificação de versões de serviços:

```
nmap -p 3000 -sV --min-rate 5000 <TARGET_IP>
```

- **Flag `-sV`:** Determina a identificação de versões de serviços ativos através de técnicas de *banner grabbing* e do envio de sondagens (*probes*) específicas de protocolos.

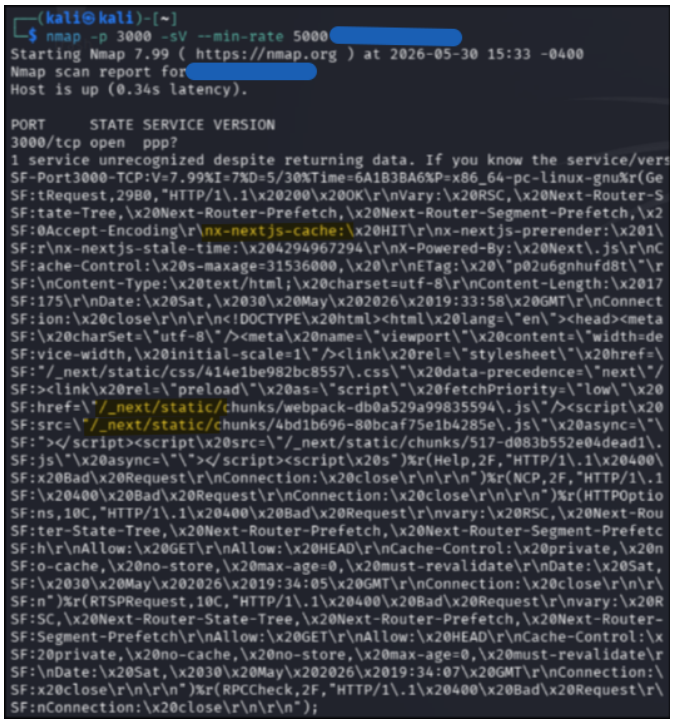


Figura 2: Enumeração detalhada da porta 3000: banner HTTP capturado com estrutura Next.js visível.

O *output* gerado pelo escaneamento trouxe um extenso *fingerprint* de resposta HTTP. A análise minuciosa do corpo HTML retornado expôs caminhos de diretórios altamente padronizados:

```
/_next/static/css/...
/_next/static/chunks/...
```

A identificação inequívoca do padrão estrutural `/_next/` confirma que a porta 3000 hospeda uma aplicação construída sobre o *framework* **Next.js**, exposta diretamente à rede sem a proteção ou intermediação de um proxy reverso. A acessibilidade pública do diretório `_next/static/` indica que os recursos

estáticos (*assets*) e os *chunks* compilados do Webpack estão expostos, criando uma superfície relevante para a enumeração manual de rotas e mapeamento de potenciais *endpoints* de API.

Superfície de Ataque Consolidada:

- **Porta 22 (SSH):** Canal para acesso e controle remoto seguro do ambiente, dependente de credenciais.
- **Porta 3000 (Next.js):** Alvo prioritário definido para a fase de exploração de vulnerabilidades web.

3 Exploração

3.1 Identificação da Versão do Next.js

Com a presença da aplicação web confirmada na porta 3000, o próximo passo foi identificar a versão exata do *framework* em execução para mapeamento de possíveis vulnerabilidades. Uma técnica simples e eficaz é a inspeção através do console de ferramentas de desenvolvedor do navegador, uma vez que o Next.js costuma expor a sua versão na propriedade do objeto global `window.next`:

```
window.next?.version
```

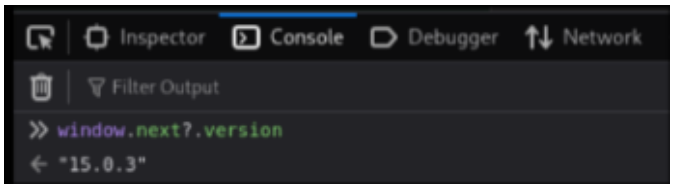


Figura 3: Console do navegador revelando a versão 15.0.3 do Next.js em execução.

O *output* retornou a versão **15.0.3**. Cruzando essa informação com bancos de dados de vulnerabilidades conhecidas (*Exploit-DB*, NVD) para o respectivo *framework*, foi possível constatar que a versão implantada se enquadra exatamente no *range* de versões afetadas pela grave falha CVE-2025-66478.

3.2 A Vulnerabilidade – CVE-2025-66478 (React2Shell)

Trata-se de uma vulnerabilidade crítica, avaliada com **CVSS 10.0**, residente no protocolo *React Server Components* (RSC). A falha permite Execução Remota de Código (RCE) no momento em que o servidor processa requisições maliciosas.

O servidor Next.js falha ao desserializar os dados recebidos na requisição sem a validação ou sanitização adequada, permitindo que *payloads* arbitrários sejam injetados e executados diretamente dentro do contexto do processo do Node.js subjacente. Originada nativamente no projeto *upstream* do React (catalogada como CVE-2025-55182), a vulnerabilidade afeta gravemente aplicações Next.js que utilizam o *App Router* nas ramificações 15.x e 16.x.

3.3 Exploração – react2shell-ultimate

Para confirmar e explorar a vulnerabilidade, utilizou-se o exploit público denominado `react2shell-ultimate`, desenvolvido por *Satyam Rastogi* e disponibilizado abertamente no GitHub.

Execução do Exploit

O *script* foi executado com parâmetros específicos para testar e comprometer o alvo imediatamente. A *flag* `--god` ativa o chamado *God Mode*, que habilita a injeção irrestrita para execução completa de comandos no servidor alvo. A *flag* `--shell` orienta o *exploit* a iniciar uma sessão interativa (*pseudo-shell*) diretamente após a confirmação da explorabilidade:

```
python3 react2shell-ultimate.py --god -u http://<TARGET_IP>:3000 --shell
```



Figura 4: Shell interativa obtida via react2shell-ultimate; confirmação de acesso ao alvo e contexto do usuário.

O *output* interativo demonstra o comprometimento bem-sucedido. O sistema retornou que o processo explorado está rodando sob os privilégios do usuário de serviço `node` (`uid=999`). Esse resultado indica que o *foothold* (ponto de apoio inicial) foi obtido em um contexto não privilegiado, demandando uma futura fase de enumeração local e movimentação lateral ou escalada de privilégios para obtenção de acesso de superusuário (`root`).

4 Pós-Exploração – Enumeração Local

4.1 Descoberta do Banco de Dados

Com a *shell* obtida via `react2shell-ultimate`, realizou-se uma listagem básica dos arquivos no diretório corrente da aplicação. Entre os arquivos retornados pelo sistema, o `reactor.db` chamou a atenção por se tratar de um banco de dados SQLite posicionado diretamente no diretório raiz do projeto.

Para explorar seu conteúdo, utilizou-se o cliente de linha de comando do SQLite para listar as tabelas existentes:

```
sqlite3 reactor.db ".tables"
```

O comando revelou a existência de duas tabelas principais: `sensor_logs` e `users`. A tabela de usuários foi consultada em seguida para extração de dados:

```
sqlite3 reactor.db "SELECT * FROM users;"
```

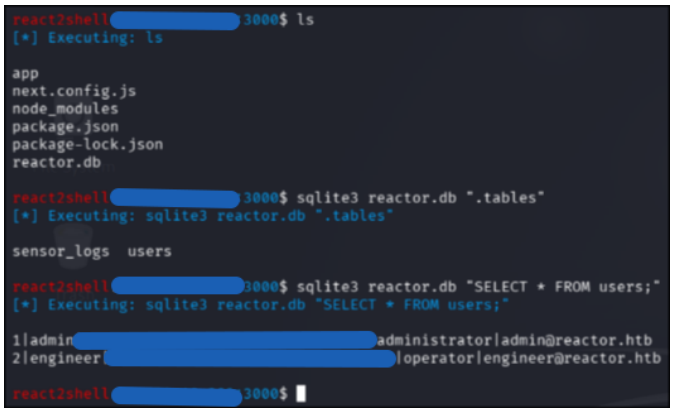


Figura 5: Listagem do diretório da aplicação e dump da tabela `users` via `sqlite3`, revelando dois registros com hashes.

O *dump* retornou dois usuários cadastrados no sistema com seus respectivos *hashes* de senha, conforme organizado na Tabela 2:

Tabela 2: Registros extraídos da tabela de usuários do banco de dados.

ID	Usuário	Hash	Role	E-mail
1	admin	`<HASH_ADMIN>`	administrator	admin@reactor.htb
2	engineer	`<HASH_ENGINEER>`	operator	engineer@reactor.htb

4.2 Identificação do Tipo de Hash

Antes de prosseguir com ataques de força bruta ou dicionário, utilizou-se o utilitário `hashid` para identificar estatisticamente o algoritmo de *hashing* empregado:

```
hashid -m '<HASH_ADMIN>'
```

O comprimento exato de 32 caracteres hexadecimais e o *output* gerado pelo `hashid` apontaram fortemente para o algoritmo **MD5** como a opção mais provável.

4.3 Quebra de Hash – John the Ripper

Com o tipo de *hash* identificado, os valores extraídos foram salvos localmente em um arquivo de texto (`hashesReactor.txt`) e submetidos ao *software* de quebra de senhas **John the Ripper**, utilizando a *wordlist* padrão `rockyou.txt`:

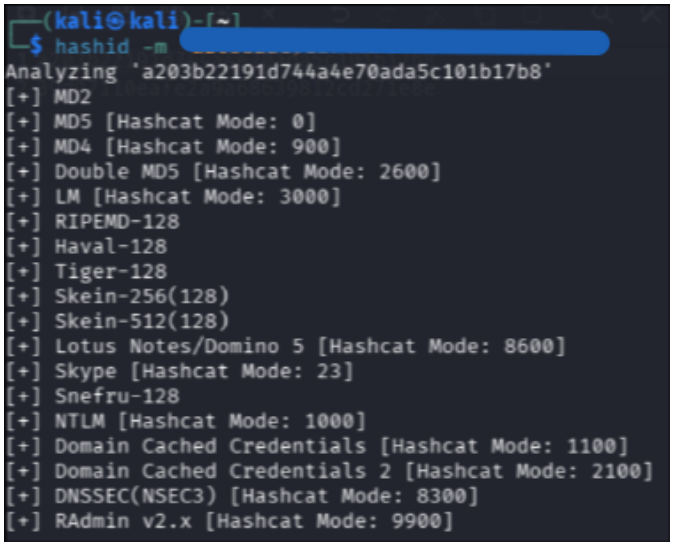


Figura 6: `hashid` identificando os possíveis algoritmos; MD5 (Hashcat Mode 0) aparece como principal candidato.

```
john --format=Raw-MD5 --show hashesReactor.txt
```

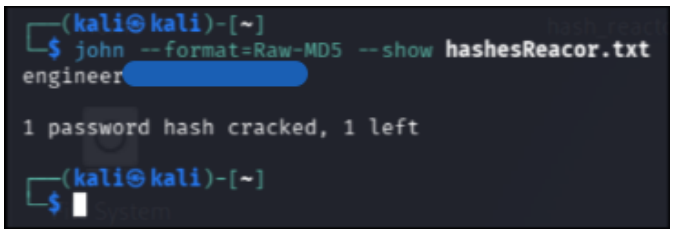


Figura 7: John the Ripper quebrando o hash do usuário `engineer`; hash do `admin` permanece sem crack.

O resultado do ataque de dicionário foi parcial: apenas a credencial do usuário `engineer` foi revertida com sucesso, conforme demonstrado na Tabela 3. O *hash* do administrador demonstrou ser resistente ao dicionário utilizado.

Tabela 3: Resultado da quebra de hashes.

Usuário	**Senha**
engineer	`<PASSWORD_ENGINEER>`
admin	(não quebrado)

Com as credenciais válidas `engineer:<PASSWORD_ENGINEER>` em mãos, o próximo passo lógico na cadeia de ataque é utilizá-las para acessar o serviço remoto SSH, o qual havia sido previamente identificado na porta 22 durante a fase inicial de reconhecimento.

5 Movimentação Lateral – Acesso SSH

Com as credenciais válidas obtidas através da etapa de quebra de *hashes*, foi possível estabelecer uma conexão remota e interativa via protocolo SSH atuando sob o contexto do usuário `engineer`:

```
ssh -p 22 engineer@<TARGET_IP>
```

A autenticação foi aceita pelo servidor utilizando a senha `<PASSWORD_ENGINEER>`, garantindo uma sessão estável de terminal.

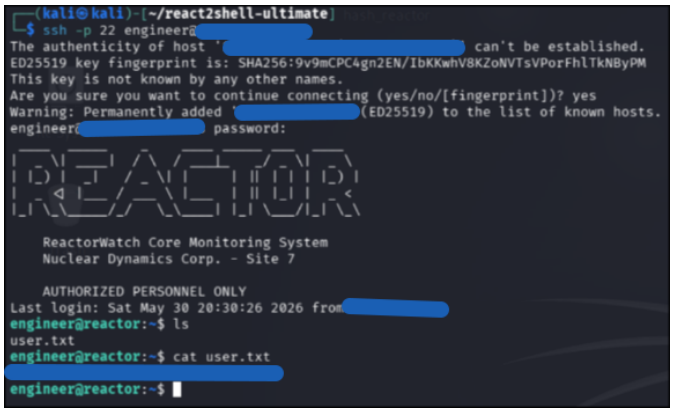


Figura 8: Login SSH bem-sucedido como `engineer`; banner do sistema e captura da flag de usuário.

Uma verificação direta no diretório *home* do usuário revelou a presença do arquivo `user.txt`. A leitura do arquivo confirmou a obtenção do primeiro objetivo do desafio:

```
cat user.txt
<FLAG>
```

A *flag* de usuário foi capturada com sucesso. O acesso atual encontra-se consolidado no sistema como `engineer` — um usuário de operação padrão, sem privilégios administrativos elevados. A próxima etapa demandará uma enumeração local minuciosa no sistema alvo em busca de um vetor factível para a escalada de privilégios até o superusuário (`root`).

6 Escalada de Privilégios

6.1 Enumeração Local – Serviços Internos

Com acesso inicial ao sistema garantido como `engineer`, o primeiro passo da pós-exploração foi mapear as conexões de rede e serviços em execução localmente (*localhost*) que não estavam visíveis para a rede externa durante o reconhecimento inicial:

```
netstat -tulpn
```

A saída do utilitário `netstat` revelou uma porta de escuta altamente relevante vinculada exclusivamente à interface de *loopback*, detalhada na Tabela 4:

Em seguida, buscou-se identificar o processo exato responsável por manter essa porta aberta através do utilitário `ps`:

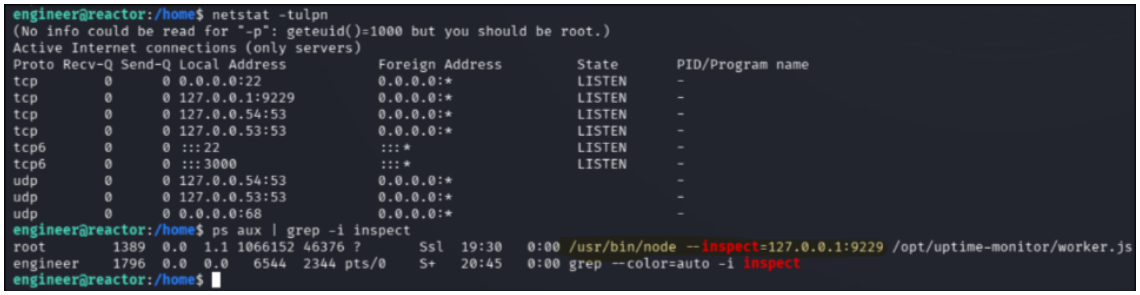


Figura 9: `netstat` revelando a porta 9229 escutando em *localhost*; `ps aux` confirmando processo Node.js rodando como root.

Tabela 4: Serviço interno identificado em estado de escuta (LISTEN).

Endereço	Porta	Protocolo
127.0.0.1	9229	TCP

```
ps aux | grep -i inspect
```

O *output* revelou a execução do seguinte comando pelo sistema:

```
root  1389  /usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/
   worker.js
```

Trata-se de um processo do Node.js rodando nativamente sob o contexto do usuário `root`, invocado com a flag `--inspect` habilitada na porta 9229. Este cenário configurou o vetor ideal para a escalada de privilégios.

6.2 O Vetor – Node.js Inspector Abuse

A flag `--inspect` instrui o ambiente de execução do Node.js a ativar o protocolo *Chrome DevTools Protocol* (CDP), expondo uma interface administrativa de *debugging* operada via *WebSocket*.

Uma vez que o *debugger* possui controle e acesso total ao ambiente de execução do processo atrelado a ele, qualquer cliente que consiga estabelecer conexão com a porta 9229 recebe a capacidade de executar código JavaScript arbitrário. Como o processo alvo foi iniciado pelo superusuário, os comandos disparados rodam invariavelmente sob o contexto de `root`. A restrição de vínculo ao `127.0.0.1` foi contornada pelo fato de já possuirmos acesso *shell* interativo à máquina via SSH.

6.3 Exploração – RCE via Node Inspector

A conexão ao *debugger* remoto foi estabelecida diretamente utilizando o cliente nativo fornecido pelo próprio Node.js:

```
node inspect localhost:9229
```

Com o console interativo de depuração aberto (`debug>`), a execução de comandos a nível de sistema operacional (*OS Command Execution*) é feita importando o módulo nativo `child_process` e utilizando a diretiva `exec`:

```
// Sintaxe basica no console debug> do inspector:
exec process.mainModule.require('child_process').execSync('comando').
   toString()
```

Com o potencial crítico de executar comandos como `root` confirmado, injetou-se o comando final em JavaScript estruturado para ler o arquivo da *flag* restrita e retornar seu conteúdo formatado como *string* diretamente no console de depuração:

```
exec process.mainModule.require('child_process').execSync('cat /root/root.
   txt').toString()
'<FLAG>'
```

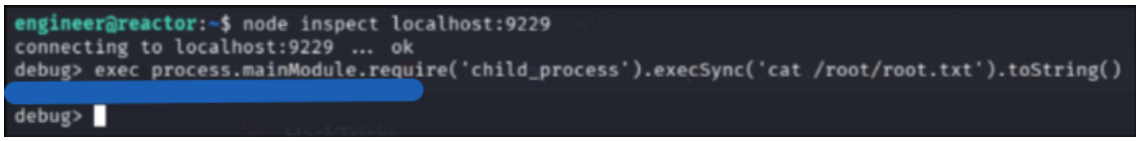


Figura 10: Acesso ao *inspector* do Node.js e comprovação da *flag* de `root` capturada com sucesso.

7 Conclusão

A máquina Reactor apresenta uma cadeia de ataque fundamentada inteiramente na exploração de tecnologias modernas do ecossistema JavaScript. O reconhecimento inicial mapeou uma aplicação Next.js 15.0.3, vulnerável à CVE-2025-66478 — uma falha crítica de *Remote Code Execution* atrelada ao *React Server Components*. A exploração dessa superfície forneceu uma *shell* restrita (`node`), abrindo caminho para a extração de credenciais de um banco de dados SQLite interno. A quebra do *hash* MD5 revelou a senha do operador `engineer`, permitindo a movimentação lateral via SSH. Por fim, o abuso de um processo `Node.js` executado impropriamente por `root` com a interface de depuração (`--inspect`) habilitada localmente permitiu a injeção de comandos via *Chrome DevTools Protocol*, resultando no comprometimento total da infraestrutura.

8 Referências

- **CVE-2025-66478 – Next.js Security Advisory:**
  https://nextjs.org/blog/CVE-2025-66478
- **react2shell-ultimate – Exploit público (Satyam Rastogi):**
  https://github.com/hackersatyamrastogi/react2shell-ultimate
- **HackTricks – Node Inspector / CEF Debug Abuse:**
  https://hacktricks.wiki/en/linux-hardening/privilege-escalation/electron-
  html